

Vivez l'aventure!

SAE 2.02 – Comparaison algorithmique d'un problème

Informatique – IUT d'Orléans

2025-2026

Introduction

Féru·e de jeux en tout genre, la collection de livres-jeu [Vivez l'aventure!](#) a bien sur bercé votre enfance. Vous vous souvenez encore avec une émotion à peine voilée de ces heures de réflexion intense afin de terminer chaque livre!

Les années ont passé et vous êtes désormais un informaticien expérimenté, issu d'une des meilleures formations française du domaine (oui oui rien que ça) : le BUT Informatique de l'IUT d'Orléans. Vous décidez de mettre à profit votre expertise afin de proposer une application permettant de modéliser ces livres-jeu qui vous tiennent tant à cœur.

Présentation du concept

L'idée est très simple : vous démarrez à la première page du livre et à chaque page, vous avez plusieurs choix de pages pour continuer. L'idée est donc de **trouver son chemin** jusqu'à la sortie, qui correspond à une page particulière du livre (qui n'est pas nécessairement la dernière). Une fois cette page trouvée, vous avez gagné!

Cependant, les choses ne sont pas si simples en pratique... De nombreuses pages **vous ramènent en fait au début**, ralentissant votre progression. De plus, chaque page contient **des énigmes** qui doivent être résolues avant de pouvoir continuer. Les énigmes sont plus ou moins dur à résoudre. Pour corser encore un peu plus votre aventure, il faut **collecter un certain nombre d'objets** se trouvant sur différentes pages, avant de pouvoir sortir.

Votre travail

Vous devez réaliser une application **java** afin de modéliser un livre-jeu. Votre application doit fournir les fonctionnalités suivantes :

- Création d'un livre-jeu avec les différentes contraintes
- Chargement et sauvegarde d'un livre-jeu en se basant sur une représentation textuelle

- Génération d'une solution (si elle existe!)
- Génération automatique d'un livre

Vous devez appliquer la **méthode TDD** lors du développement et utiliser un dépôt **Git** pour le versionnage et la collaboration.

Cependant, vous constaterez vite qu'il vous faudra faire des **choix**, qu'il soit algorithmique ou d'implémentation (structure de données). Il vous est donc demandé de démontrer la pertinence de vos choix et la qualité de votre solution en procédant à une **évaluation expérimentale**.

On s'intéressera aux performances de la recherche de solution et de la génération d'un livre-jeu.

Recherche d'une solution

Une solution pour un livre jeu correspond à la liste des pages qui doivent être lues afin d'arriver à la sortie.

Une solution est dite **valide** si :

- deux pages successives de la solution sont bien reliées dans le livre,
- la première page de la solution correspond à la première page du livre
- la dernière page de la solution correspond à la page de sortie du livre
- l'ensemble des objets collectés sur chaque page de la solution correspond à l'ensemble des objets qu'il est nécessaire de collecter pour terminer le jeu

Plusieurs mesures de performance seront étudiées.

- le **temps de parcours** d'une solution correspond à la somme des temps de résolutions des énigmes de chaque page
- la **longueur** d'une solution correspond aux nombres de pages qu'elle contient
- le **temps d'exécution** (en seconde) nécessaire pour calculer une solution

Vous devez proposer **au moins deux algorithmes** de recherche de solution que vous comparez sur les mesures de performance évoquées plus haut et sur différents livres-jeu.

Génération d'un livre-jeu

On veut pouvoir générer un livre-jeu à partir des paramètres suivants :

- Le nombre total de pages
- La page de début
- La page de sortie

- Les emplacements des objets à récupérer
- Les énigmes pour chaque page (un texte autogénéré de type lispum suffit) et le temps de résolution estimé

On ajoute la contrainte que toute page du livre doit pouvoir être atteinte depuis la page de départ et qu'il est possible d'accéder à la sortie depuis n'importe quelle page (quitte à revenir au début de l'histoire pour cela).

Pour un même paramétrage, on aimerait pouvoir générer différents livres-jeu qui diffèrent par les liens entre les pages.

Vous devrez proposer **au moins deux algorithmes de génération** et vous les comparerez par rapport à leurs temps d'exécution.

Les livrables

Le travail sera réalisé en **groupe de 4**. Les groupes seront **identiques** à ceux de la SAE Briqu'IUT'O.

La modélisation → 10 avril

Vous devez rédiger un **premier rapport** présentant :

- Les informations concernant votre binôme ainsi que le lien vers votre dépôt github.
- La modélisation du problème que vous proposez. Vous devez utiliser les notions vues en graphes pour cela.
- Un premier diagramme de classes de votre application. Ce diagramme sera sûrement amener à changer au cours du développement, mais il faudra alors justifier ces évolutions.

L'implémentation → 29 mai

Vous devez remettre dans une **archive zip ou tar.gz** (et pas d'autre format!) les documents suivants :

- un rapport présentant votre application et surtout :
 - la **justification** de la correction de vos algorithmes
 - l'**évaluation expérimentale** des différents algorithmes que vous avez considérés. Le protocole expérimental devra être décrit avec précision
 - un **manuel détaillé** permettant de compiler et d'exécuter votre application. On vous demande d'utiliser **maven** pour le projet.
- le code source de votre application
- une archive jar de votre application prête à être exécutée.

Evaluation du module

Vous serez évalués sur :

- les **livrables** que vous aurez remis
- une **évaluation individuelle** sera également mise en place, dont la modalité vous sera précisée ultérieurement (QCM ou DS sur table).
- Pour finir, une **soutenance** sous forme d'une démonstration de votre application viendra compléter l'évaluation de cette SAE. Cette soutenance aura lieu la semaine du **1er juin 2026**.

Note sur l'usage de l'IA

L'utilisation d'IA génératives est **fortement déconseillée**. Si vous y avez recours vous devez l'utiliser exclusivement pour apprendre et non pour faire à votre place. **Tout groupe faisant passer pour son travail un contenu (même mineur) généré par une IA sera sanctionné par un zéro.**